# PCA Fundamentals

Krishna Sekhar
Padma Venkatraman

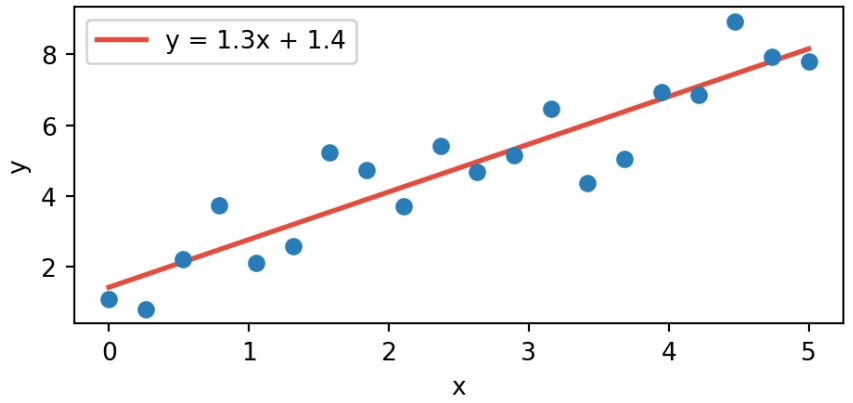
# Recap

- Before we go further **remember** :
  - Covariance : Measures how two variables change with each other
  - Eigenvectors and eigenvalues : An eigenvector is a direction that a matrix only stretches, never rotates — and the eigenvalue is how much it stretches.
  - Matrix operations are in general **linear transformations**. Meaning they tend to stretch and/or rotate whatever they are acting upon.

# Fitting and Prediction

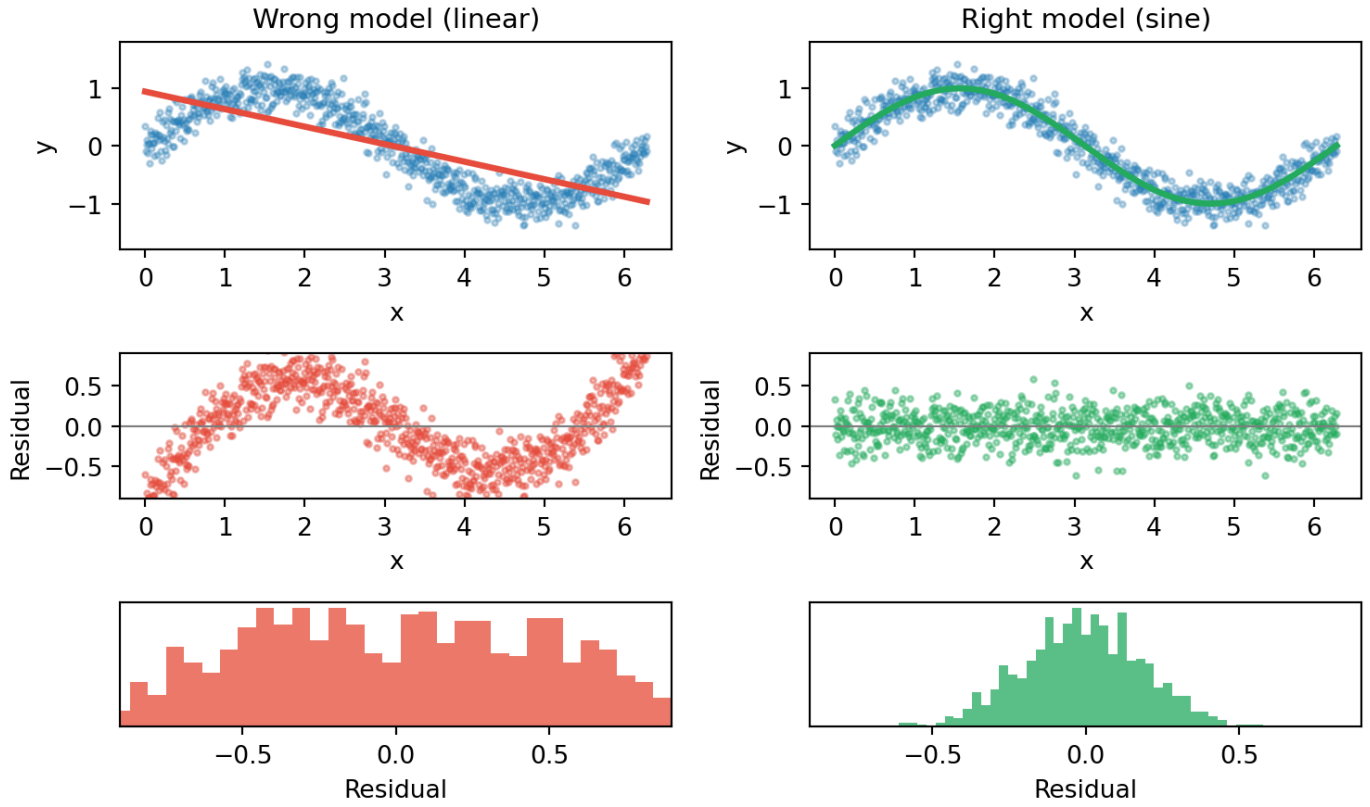
# What is fitting?

- You have data and you have a model (e.g., $y = mx + b$)
- Fitting = finding the parameters ($m$, $b$) that make the model best describe the data
- The **best fit** curve is a model representation of the data



# Choosing the right model

What if we pick the wrong model? If residuals still have structure, the model is wrong — residuals should look like **noise**.



# Fitting vs. Prediction

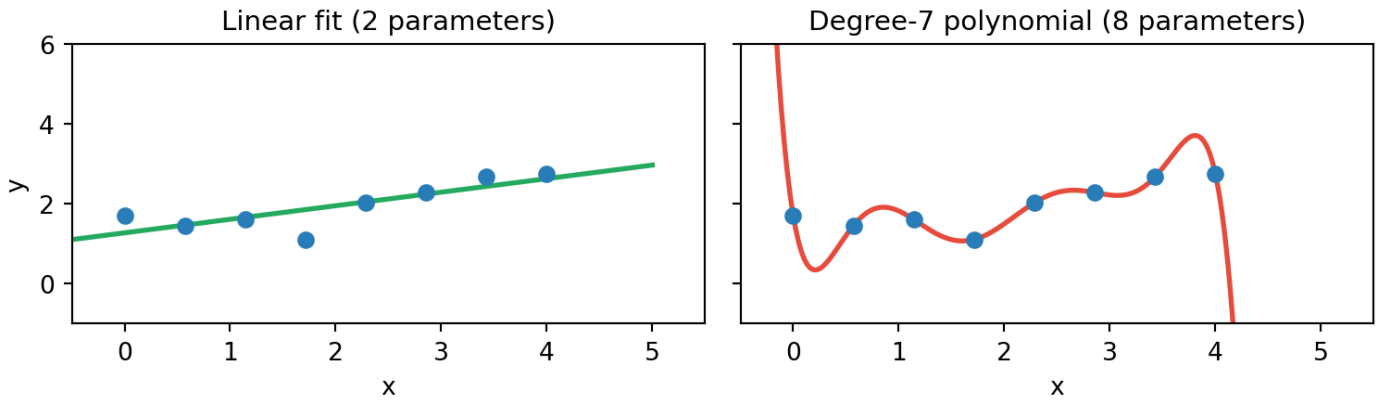
- **Fitting** — how well does the model describe the data you *have*?
- **Prediction** — how well does it work on data you *haven't seen*?
  - *Interpolation* — predicting *between* known data points
  - *Extrapolation* — predicting *beyond* the range of your data

> **Note**
>
> A model that fits perfectly can still predict terribly.

# Overfitting

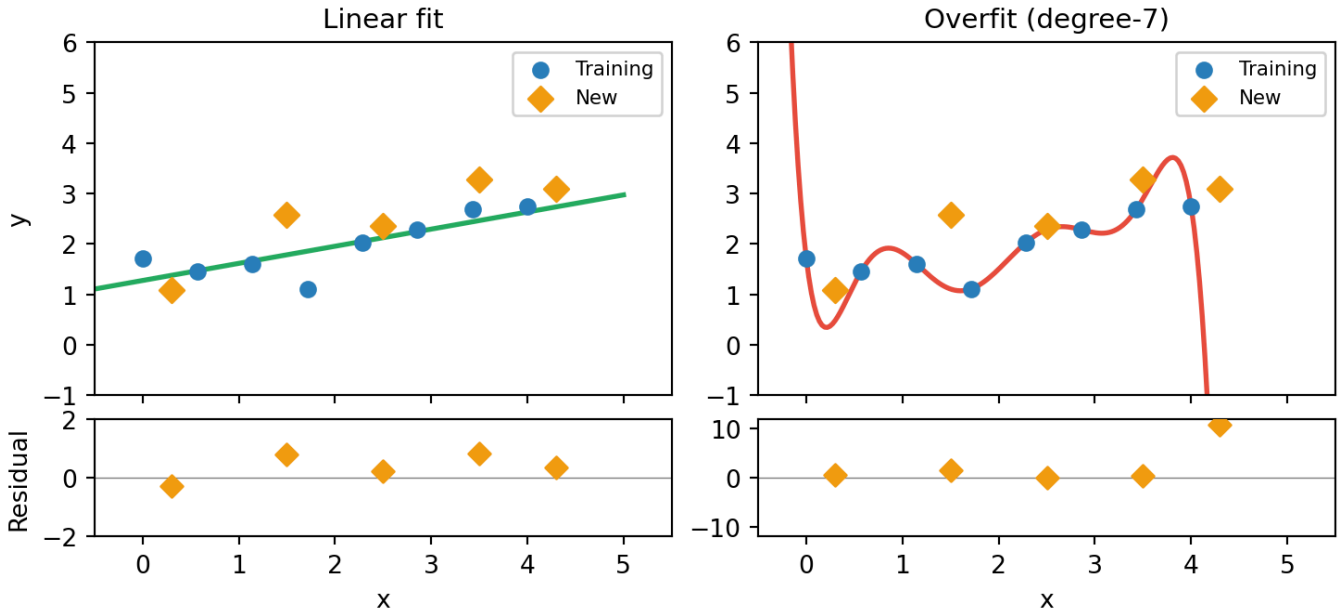
**Discuss**

The degree-7 curve passes through every point with zero error. Is it the better model?



# How do you spot overfitting?

Fits the training data beautifully — but the new data residuals give it away.

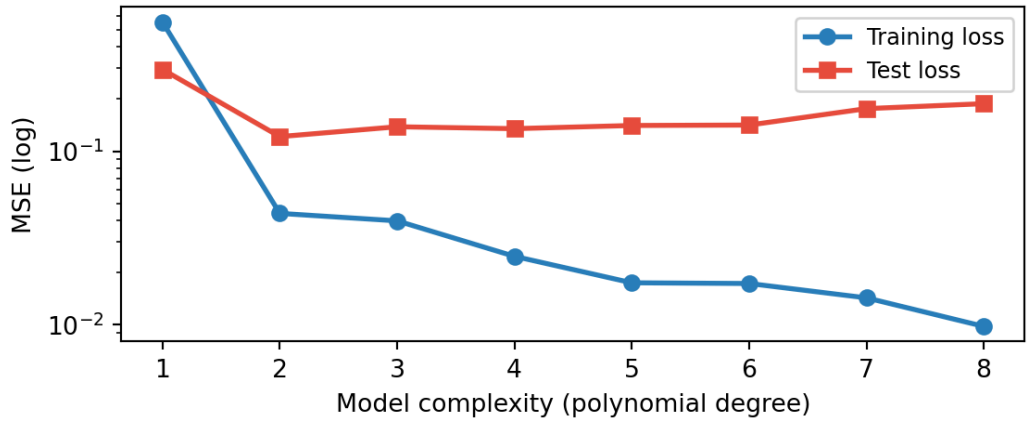



Linear residuals are small; overfit residuals blow up

# Train / Test split

What if we **held some data back** on purpose?

- Split your data: *train* on one portion, *test* on the other
- Fit the model on training data only
- Evaluate on the held-out test data
- If test error ≫ training error ⟶ you're **overfitting**



# Fitting on an unfamiliar dataset

When fitting we try to **estimate** the signal given a noisy data set.

Underlying assumption : We *know* which *features* matter, and we can run a fit on those select features.

Problem : Take a sky survey catalog, we have millions of sources with several properties each : redshift, size, flux, luminosity, SNR, spectra etc.

How do we know which handful of features are relevant to our science question? And can we drop the rest?

# Principal Component Analysis (PCA)

# What & Why

- **What**
  - Find the axes along which the data shows highest variation, and we can drop the rest.
- **Why**
  - Astronomy (and many other) datasets often have many tens to hundreds of dimensions (mass, redshift, flux, brightness, type, SNR, spectra etc.)
  - We want to find the variables that **matter**, and discard the other (distracting) ones.

# PCA - contd.

For e.g., say we want to classify the sources in a continuum image, and we have flux, luminosity, redshift, and SNR as measurements…

luminosity is **not independent** — it is determined by flux and redshift

Redundant dimensions add noise, slows down analysis and makes visualization very challenging.

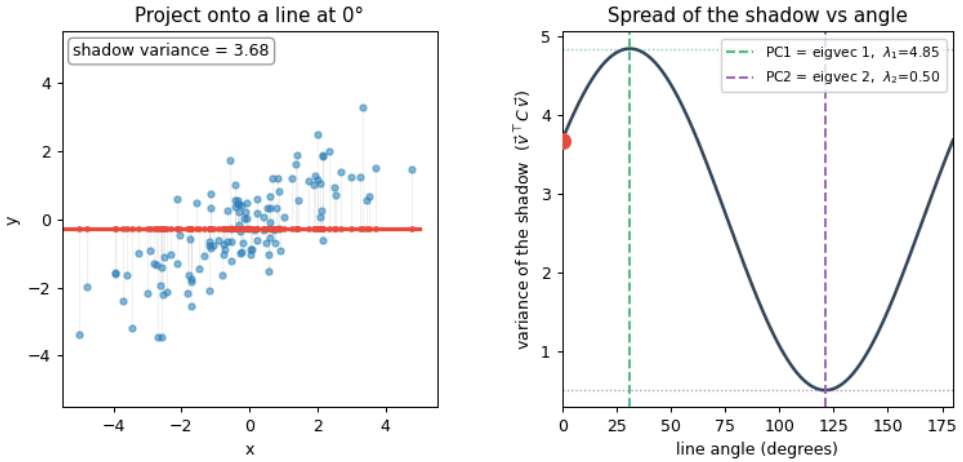
> **Note**
>
> Identifying which dimensions matter is often challenging. PCA offers a method to automatically do it.

# Eigenvectors as Principal Components

- The covariance matrix captures how your variables co-vary
- Its eigenvectors point in the directions of **maximum variance** in the data
- These directions *are* the principal components
- The eigenvalue tells you how much variance each PC captures
- PC1 = largest eigenvalue, PC2 = next largest, …

# PCA: the line of maximum spread

Spin a line through the data and project every point onto it. The angle where the **shadow spreads out most** is PC1 — and that line is exactly the *top eigenvector* of the covariance matrix.




The peak spread *is* the eigenvalue $\lambda_1$; the perpendicular direction (least spread) is PC2, with eigenvalue $\lambda_2$.

# PCA in five steps

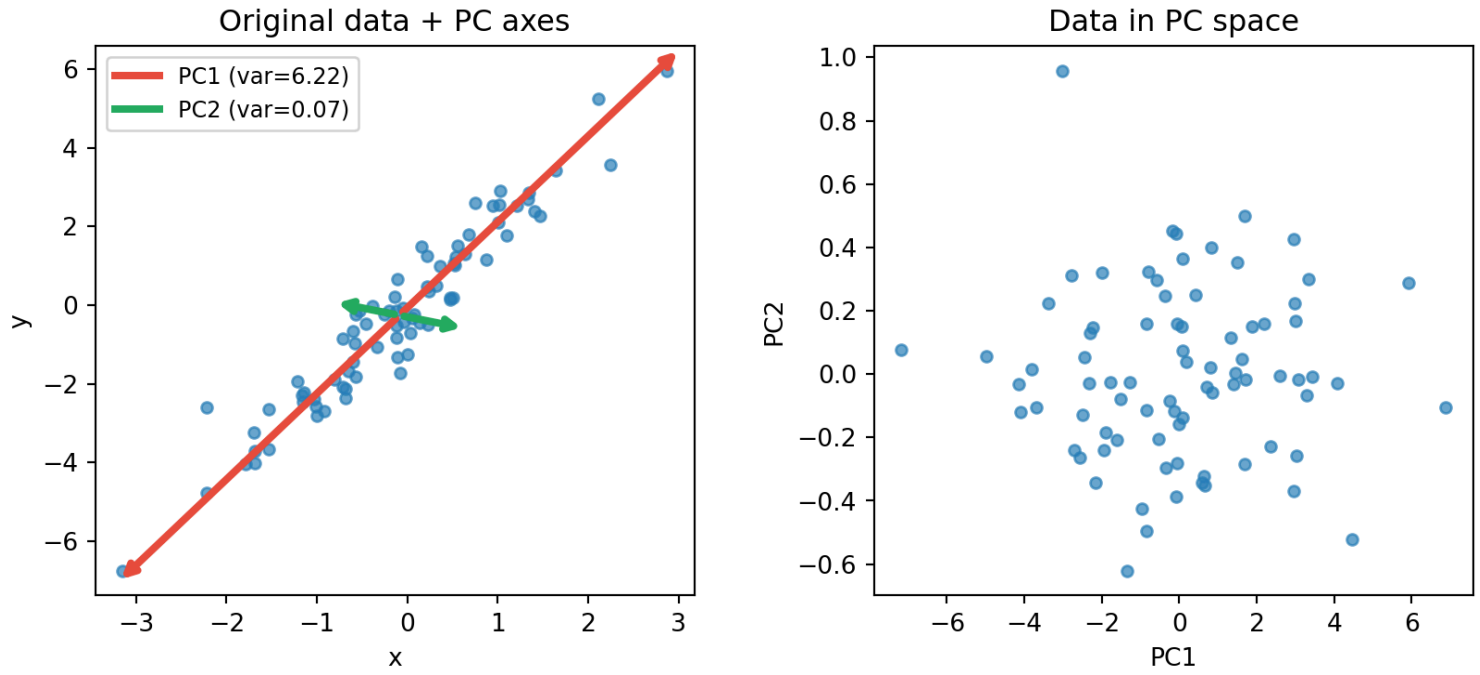
| Step | What it does | Who does it |
|---|---|---|
| **Center** | shift the cloud to the origin | automatic |
| **Covariance** | capture the cloud's shape | automatic |
| **Eigendecompose** | find its natural axes | `np.eigh` — automatic |
| **Rank & keep** | order axes by spread, keep top $k$ | **you pick $k$** |
| **Project** | re-coordinate points on those axes | automatic |

The only judgment call is **how many components to keep** — everything else is typically done by the algorithm implementation.

If you're doing this by hand, you'll need to do all the steps above!

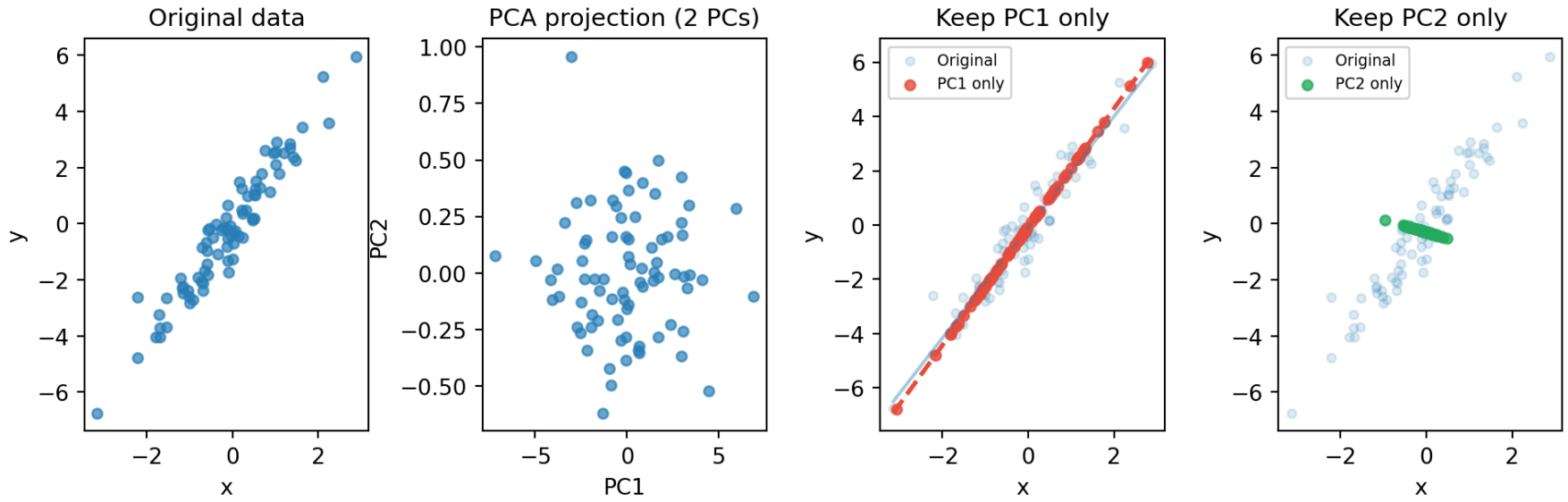
The appendix works through each step on a 6-point toy dataset (self-study).

# PCA Visual Intuition




PC1 captures most of the spread — PC2 is almost just noise.

# Dropping Dimensions




Dropping PC2 barely changed the fit — PCA found the $y \approx 2x$ relationship without being told the model.

# When is dropping a PC a bad idea?

**Discuss**

Here PC2 was noise — when would throwing away the low-variance direction hurt you?

# What's *inside* a PC?

The PC direction is the *eigenvector* $\vec{v}$ of the covariance $C$ — the *same* $\vec{v}$ as in $C\vec{v} = \lambda\vec{v}$ from before.

A data point $\vec{x}$ (one spectrum) projected onto it gives that point's *score*:

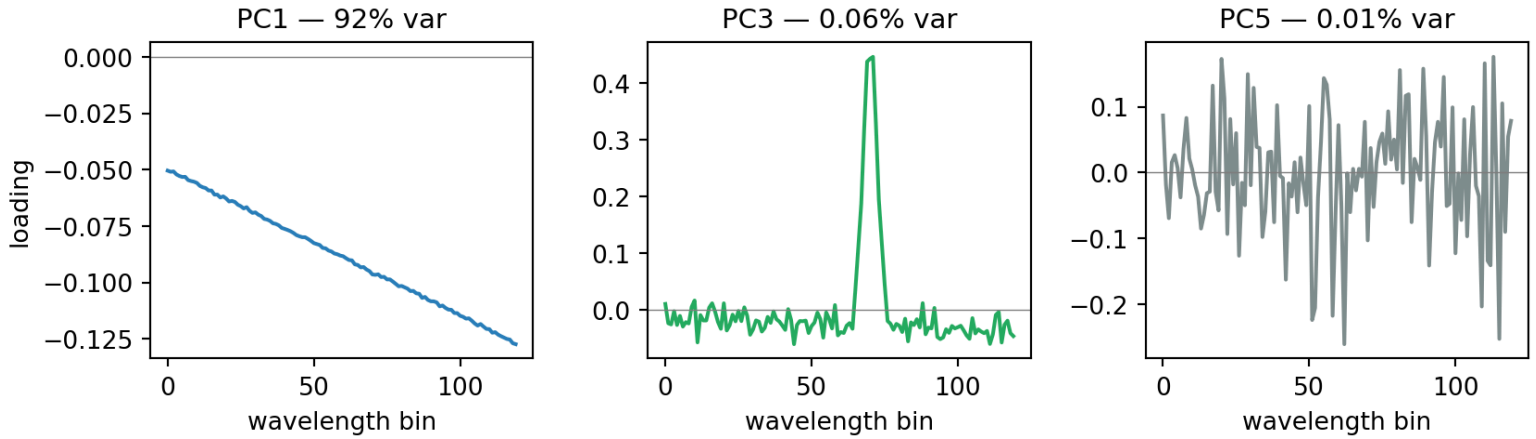
$$z = \vec{v} \cdot \vec{x} = \underbrace{v_1}_{\text{loading}} x_1 + v_2 x_2 + \cdots + v_p x_p$$

$\vec{x}$ = a *data point*, $v_i$ = the *loadings* (eigenvector entries, one per feature), $z$ = *score*.

Eigenvalue $\lambda$ = variance of $z \rightarrow$ **how much** a PC matters; loadings $v_i \rightarrow$ **what** it is about.

# Reading the loadings

Tell a *meaningful* PC from a noise one by plotting its *loadings* $v_i$ vs feature index — 400 spectra, 120 wavelength bins. Structure in the $v_i$ = the PC responds to *specific* wavelengths.




Structured loadings = signal (PC3's **spike at the line**, despite tiny variance); random = noise (PC5).

# Appendix — (worked on a 6-point toy dataset; for self-study) the PCA math

# PCA step by step — find the axes

Toy data: 6 points in 2D. Stack them as rows of a matrix (one column per feature):

$$X = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 3 & 2 \\ 5 & 4 \\ 4 & 3 \\ 1 & 0 \end{bmatrix} \xrightarrow[\bar{X} = (2.5,\ 1.67)]{\textbf{1. center}} X_c = X - \bar{X}$$

**2. covariance:** $C = \frac{1}{n} X_c^\top X_c = \begin{bmatrix} 2.92 & 2.17 \\ 2.17 & 2.22 \end{bmatrix}$

The off-diagonal is large and positive → $x$ and $y$ rise together.

# PCA step by step — eigendecompose

**3. Eigendecompose** — apply $A\vec{v} = \lambda\vec{v}$ from lecture 1a to the covariance matrix $C$.

Stack all the eigenvectors as columns of $V$ and the eigenvalues on the diagonal of $\Lambda$. Then the per-vector relation $C\vec{v}_i = \lambda_i\vec{v}_i$ holds for every $i$ at once:

$$C\vec{v}_i = \lambda_i\vec{v}_i \qquad \Longrightarrow \qquad CV = V\Lambda$$

Since the eigenvectors are orthogonal, $V^\top = V^{-1}$ , which rearranges to the diagonalisation

$$C = V\Lambda V^\top$$

# PCA step by step — the eigenpairs

For our toy $C$, this gives two eigenpairs:

$$\lambda_1 = 4.76, \quad \vec{v}_1 = \begin{bmatrix} 0.76 \\ 0.65 \end{bmatrix} \qquad \lambda_2 = 0.38, \quad \vec{v}_2 = \begin{bmatrix} -0.65 \\ 0.76 \end{bmatrix}$$

The eigenvector $\vec{v}_i$ is a **direction**; its eigenvalue $\lambda_i$ is the **variance of the data along that direction**.

**4. Sort** by $\lambda$, largest first $\rightarrow$ $\vec{v}_1$ is PC1, $\vec{v}_2$ is PC2. The eigenvectors are orthogonal.

# PCA step by step — project

**5. Project** onto the new axes — $Z = X_c V$:

- $V$ — the eigenvector matrix, **PCs as its columns** (from step 3)
- $Z$ — the *scores*: each row is one point's coordinates in the PC axes

Each point is re-described by **how far it lies along each PC**, instead of by its original $(x, y)$.

Why bother? The PCs are ranked by variance and uncorrelated — so the *useful* spread is front-loaded into the first few columns of $Z$, and the axes no longer redundantly encode the same trend.

**6. Variance explained** — $\lambda_i / \sum_j \lambda_j$:

$$\text{PC1} : \frac{4.76}{4.76 + 0.38} = 92.7\% \qquad \text{PC2} : 7.3\%$$

# PCA step by step — drop a dimension

"Dropping PC2" = **keep only the first $k$ score columns** and discard the rest:

$$Z_{\text{full}} = X_c V \quad (n \times 2) \qquad \longrightarrow \qquad Z_k = X_c V_{:,1} \quad (n \times 1)$$

Each point now needs **one number, not two** — geometrically, you flatten the cloud onto the PC1 axis.

To view it back in the original space, reconstruct: $\hat{X} = Z_k V_{:,1}^{\top} + \bar{X}$.

**One axis carries 93% of the variance** — so this loses almost nothing.